

Notification System Design Study Notes

Overview of Notification Systems

* **Definition:** Systems that deliver real-time alerts or updates to users across different platforms (e.g., instant messaging, financial credits, e-commerce order confirmations).

* **Scope of the Design:** Focuses purely on how the notification system receives a notification request and successfully delivers it to the end user. It does not focus on how notifications are triggered or how devices are registered.

* **Types of Notifications:**

* **Push Notifications:** Delivered directly to smartphones/apps.

* **SMS Notifications:** Sent as text messages to mobile numbers.

* **Email Notifications:** Sent to user email addresses.

Gathering System Requirements

Before designing, clarify requirements with the interviewer to determine the system's depth and scope. Key questions to ask:

* **Notification Types:** Support for all three types (Email, SMS, and Push notifications).

* **Timing:** Real-time (delivered as close to the trigger time as possible).

* **Supported Devices:** Laptops, mobile devices (Android, iOS).

* **Trigger Types:**

* **Automatic:** E.g., Order confirmations triggered by user actions.

* **Scheduled:** E.g., Daily reminders at a specific time (like 9:00 AM).

* **Manual:** Triggered on-demand by system operators.

* **User Preferences:** Option for users to opt out or cap the number of notifications received per day.

High-Level Design & Third-Party Integrations

A basic design connects a notification provider to third-party services to deliver messages to clients:

* **Email Services:** Connects to platforms like MailChimp, ActiveCampaign, or Sendinblue.

* **SMS Services:** Connects to platforms like Twilio, Nexmo, or Cinch.

* **iOS Push Notifications:** Routes through Apple Push Notification service (APNs).

* **Android Push Notifications:** Routes through Firebase Cloud Messaging (FCM).

User Data and Information Gathering

To deliver notifications, the system must first gather and store user details.

1. **Signup Flow:** The user installs the app or signs up, filling out phone numbers, emails, etc.

2. **Consent Dialog:** When a user agrees to receive push notifications, a unique **device token** is sent to the API servers.

3. **Database Storage:** The system stores details in two primary relational tables:

* **User Table:** User ID (Primary Key), Email, Phone, Country, Created At.

* **Device Table:** Device Token, User ID, Last Active.

Optimized and Reliable System Architecture

A direct connection from the notification system to third-party services creates a **Single Point of Failure (SPOF)**. If the service fails, notifications are lost.

To resolve this and optimize performance, the architecture is updated with the following components:

1. Databases and Caches

- * **Databases:** Store user information, device tokens, and **notification templates**.

- * **Caches:** Speed up message construction by quickly serving templates and user preferences.

2. Message Queues (Decoupling)

- * The notification system creates a customized message body (typically in JSON format).

- * Instead of calling third-party APIs directly, messages are routed into dedicated message queues based on category (e.g., **iOS Queue**, **Android Queue**, **SMS Queue**, **Email Queue**).

3. Workers

- * Independent workers consume messages from their respective queues and send them to the third-party delivery services.

4. Notification Log & Retry Mechanism

- * Workers maintain a log of every notification status (pending, sent, failed).

- * If a delivery fails, the worker triggers a **retry mechanism** and pushes the message back to the corresponding queue for processing when the service is healthy.

5. Rate Limiting and Security

- * **Rate Limiter:** Throttles incoming notification requests to protect the system if a source service starts spamming notifications.

- * **Authentication:** Devices must be authenticated before receiving sensitive alerts (e.g., bank account balances).

Event Tracking & Analytics

To measure the effectiveness and reliability of notifications, an analytics service connects to all components to track user interaction events:

- * **Tracking Lifecycle:**

- * ``Pending` $→$ `Sent` $→$ `Delivered` or `Error``

- * Once ``Delivered` $→$ User either `Clicks` or marks as `Spam`.`

- * **Purpose:** Helps audit system health, error rates, and overall user engagement metrics.